8. Bölüm
Diziler

8.1 Dizi Nedir

Şu ana kadar en basit **veri yapıları** (data structure) olan değişkenlerle karşılaştık. Programlamada kullanılan bir başka veri yapısı da **dizilerdir** (array). Matematikte diziler; bir sıralı elemanlardan oluşan bir listedir. Sıralı elemanların sayısına **dizinin uzunluğu** (array size) denir. Doğal olarak eleman sayısı tam sayıdır (integer). Elemanların sırasını tam sayı (integer) olan **indis** (index) belirtir. Kümelerin (set) aksine, aynı elemanlar dizide farklı konumlarda bulunabilir. Elemanlara terim adı da verilir. Örnekler;

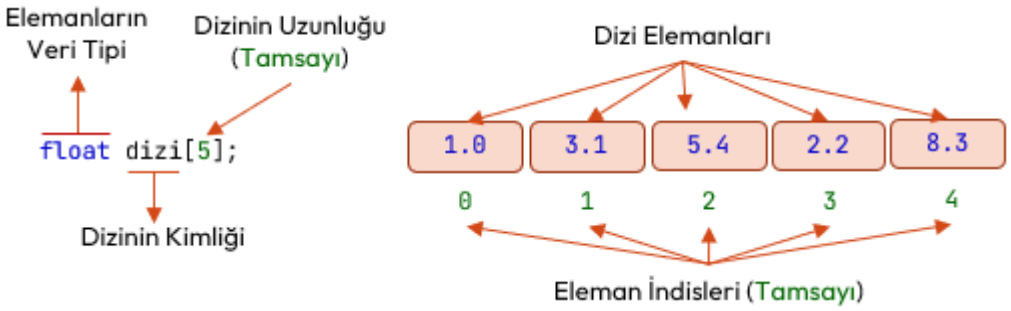
- Örnek olarak elemanları 1'den 10'a kadar tam sayıların karesinden oluşan bir dizi, {1,4,9,...,100} olarak gösterilir.
- Bir başka örnek olarak {K,İ,T,A,P} dizisi verilebilir. Bu dizi ile {P,A,T,İ,K} dizisi birbirinden farklıdır. Aynı elemanlardan oluşmasına rağmen elemanların sıraları yani indisleri farklıdır.
- {1,3,5,1,2,1,7} dizisinde birden farklı sırada yani indiste aynı 1 elemanı vardır. Bu dizinin geçersiz olduğu anlamına gelmez.

Matematikte sonsuz elemanı olan sonsuz diziler tanımlanabilir. Ama programlamada hep sonlu elemanlı diziler kullanılır. Dizinin eleman sayısı, dizinin **uzunluğudur** (size) ve dizinin uzunluğu dizi tanımlanırken belirlenir!

8.2 Tek boyutlu Diziler

C++ dilinde sonlu elemanlı diziler Şekil 8.1'deki gibi tanımlanır;

- Diziler, **aynı veri tipindeki öğeleri barındırır** ve bu öğeler **bitişik bellek bölgesini paylaşırlar**.
- Dizi elemanları, ilkel veri tipleri (`char`,`short`, `int`, `long`, `float`, `double`) veya daha sonra göreceğimiz kullanıcı tanımlı tip olan yapı `struct` veya gösterici (pointer) olabilir.
- Aynı değer birkaç farklı yerde dizi içinde bulunabilir!
- Dizinin veri tipi, **dizideki elemanlarının veri tipidir**. Bir başka deyişle **elemanların veri tipi dizinin veri tipiyle aynı olmalıdır**.
- Dizinin uzunluğu (size), **tam sayıdır.** Dizi **kimliklendirmesi sırasında belirlenir**! Derleyici buna göre bellekte yer ayırır. Uzunluk, sonradan değiştirilemez!
- Dizi uzunluğu, **sabit olmayan bir değişkenle** belirlenemez!
- Elemanların indisi (index) sıra belirttiğinden **her zaman tam sayıdır**.
- İlk elemanın indisi `0`, son elemanın indisi dizi uzunluğundan (size) bir küçük olan tam sayıdır. Bunların dışında bir indis girildiğinde **sınır dışı** (out of bounds) hatası yaşanır.



Şekil 8.1: Tek Boyutlu Dizi Tanımlama

Örnek olarak beş tam sayı elemanı olan bir dizi aşağıdaki şekilde tanımlanır;

```
int tamsayiDizisi[5];
```

Benzer şekilde beş gerçek sayı elemanı olan bir dizi ise aşağıdaki şekilde tanımlanır;

```
float reelsayiDizisi[5];
```

Dizinin elemanlarına tanımlama sırasında **başlangıç değeri** (initial value) süslü parantezler arasında verilebilir yani dizi bazı değerlerle **başlatılabilir** (initialize);

```
int tamsayiDizisi1[5]= {1, 2, 3, 4, 5}; //Elemanları 1,2,3,4,5 olacak şekilde başlatıldı.
int tamsayiDizisi2[5]= {0}; //Elemanların hepsi 0 olacak şekilde başlatılıyor.
```

Dizinin elemanlarına hepsine başlangıç değeri verilmeyebilir;

```
float a[5] = {1.0, 2.0, [4] = 12.0}; // 1.Eleman 1.0, 2.Elaman 2.0 ve 5.eleman 12.0
```

Dizinin elemanlarına ayrı ayrı erişilip değerleri değiştirilebilir ya da yeni değer ataması yapılabilir;

```
int a[5]={0}; // a dizisi, tüm elemanları 0 olacak şekilde başlatılıyor
a[0]=1; //Birinci elemana 1 değeri atandı
a[1]++; //İkinci elemanın değeri artırıldı
a[2]--; //Üçüncü elemanın değeri azaltıldı
a[4]=12; //Beşinci elemana 12 değeri atandı.
```

Diziler belleği verimli kullanan ve tek bir değişken ile elemanlara erişim sağlayan bir çözüm sunar. Bir dizideki elemanlar, bellekte bitişik konumda yer aldığından herhangi bir öğeye kolaylıkla erişebiliriz. Dizilerin başlıca üstünlükleri şunlardır:

- İndisleri kullanarak dizi öğelere rastgele ve hızlı erişim sağlanır. Her öğenin bir indisi olduğundan doğrudan erişilebilir ve değiştirilebilir.
- Birden fazla öğeden oluşan tek bir dizi oluşturduğundan daha az kod satırı yazılır.
- Daha az kod satırı yazılarak sıralama yapılabilir.

Dizi kullanmanın zayıf yönleri ise;

- Kimliklendirme sırasında dizi uzunluğuna bir tam sayı ile karar verildiğinden belli sayıda elemanın üzerinde işlem yapılır.
- Dizi dinamik değildir araya elaman ekleme veya çıkarma yapılamaz.

Örnek olarak klavyeden girilen 5 adet tam sayıyı, giriş sırasının tersinden ekrana yazan C++ programını verebiliriz;

```
#include <iostream>
int main() {
    int dizi[5]; /* 5 elemanı tam sayı olan bir dizi tanımlandı*/
    int indis; /* dizi elemanlarını gezecek ve indis olarak kullanılacak tam sayı bir değişken
    ↪ tanımlandı */
    for (indis =0; indis <5; indis ++){
        std::cout << indis << ". sayiyi girin:";
        std::cin >> dizi[ indis ]; /* indis ile belirtilen elemana klavyeden okunan değer
        ↪ atanıyor */
    }
    std::cout << "Tersten Sayilar:" << std::endl;
    for (indis=4; indis >=0; indis --)
        std::cout << "dizi[" << indis << "]=" << dizi[ indis ] << std::endl;
}
/* Program Çıktısı:
0. sayiyi girin:1
1. sayiyi girin:3
2. sayiyi girin:6
3. sayiyi girin:8
4. sayiyi girin:3
Tersten Sayilar:
dizi[4]=3
dizi[3]=8
dizi[2]=6
dizi[1]=3
dizi[0]=1
*/
```

Bir başka örnek; Klavyeden girilen 10 adet sınav notuna göre, ortalamanın üstünde olan notları ekrana yazan C++ programı;

```
#include <iostream>
#define BOYUT 10
int main() {
    unsigned notlar[BOYUT]; /* Her terimi pozitif olan 10 elemanlı dizi */
    float ortalama, toplam=0;
```

```
    int indis; //indis için tam sayı değişken
    for (indis=0; indis<BOYUT; indis++){
        std::cout << indis << ". notu girin:";
        std::cin >> notlar[indis];
        toplam+=notlar[indis]; // Her eleman girildiğinde toplanıyor
    }
    ortalama=toplam/BOYUT;
    std::cout << "Ortalamanın (" << ortalama << ") Üzerindeki Notlar:"<< std::endl;
    for (indis=0; indis<BOYUT; indis++)
        if (notlar[indis]>ortalama)
            std::cout << "notlar [" << indis << "]=" << notlar[indis] << std::endl;
}
/* Program Çıktısı:
1. notu girin:85
2. notu girin:45
3. notu girin:60
4. notu girin:75
5. notu girin:40
6. notu girin:25
7. notu girin:35
8. notu girin:60
9. notu girin:50
Ortalamanın (57.5) Üzerindeki Notlar:
notlar [0]=100
notlar [1]=85
notlar [3]=60
notlar [4]=75
notlar [8]=60
*/
```

Bir başka örnek olarak 10 adet satış miktarlarının, ortalamaya olan uzaklıkları yani sapma (deviation) ve karesi alınmış sapmalar yani varyans (variance) ile standart sapmayı yazdıran aşağıdaki C++ programı verilebilir. Ortalamaya olan uzaklıklar yani sapma, verilerin ortalamaya ne kadar yakın olduğunu gösteren dağılımdır. Sapmaların toplamı sıfır olabileceğinden karelerinin toplamı yani varyans hesaplanır. Varyansın eleman sayısına bağlı karekökü de standart sapmayı verir. Standart sapmanın büyük olması verilerin ortalamadan daha uzak yayıldıklarını; küçük bir standart sapma ise verilerin ortalama etrafında daha yakın gruplaştıklarını gösterir.

```
#include <iostream>
#include <iomanip>  // setw() ve setprecision için
#include <cmath>
#define BOYUT 10
int main() {
    float satislar[BOYUT]={100.0,850.0,500.0,600.0,750.0,650.0,450.0,800.0,900.0,110.0};
    float ortalama, toplam=0, varyansToplam=0, standartSapma;
    int indis;
    for (indis=0; indis<BOYUT; indis++)
    toplam+= satislar[indis];
    ortalama=toplam/BOYUT;
    std::cout << "Ortalama:" << ortalama << std::endl;
    for (indis=0; indis<BOYUT; indis++) {
        float sapma= satislar[indis]-ortalama;
        varyansToplam+=sapma*sapma;
        std::cout << "Sapma: " << std::setw(5) << sapma << " Varyans: " <<  std::setw(8) <<
         ↪  sapma*sapma << std::endl; /* setw(5): izleyen değişken konsola yazılırken 5 karakter
         ↪  genişliğinde yazılsın. setw(8): izleyen değişken konsola yazılırken 8 karakter
         ↪  genişliğinde yazılsın */
    }
    standartSapma=sqrt(varyansToplam/BOYUT);
    std::cout << "Standart Sapma:"<< std::setprecision(8) << standartSapma; /* setprecision(8):
     ↪  izleyen reel sayı değişkeni konsola yazılırken 9 karakter genişliğinde yazılsın. */
}
/* Program Çıktısı:
```

```
Ortalama:571
Sapma:  -471 Varyans:   221841
Sapma:   279 Varyans:    77841
Sapma:   -71 Varyans:     5041
Sapma:    29 Varyans:      841
Sapma:   179 Varyans:    32041
Sapma:    79 Varyans:     6241
Sapma:  -121 Varyans:    14641
Sapma:   229 Varyans:    52441
Sapma:   329 Varyans:   108241
Sapma:  -461 Varyans:   212521
Standart Sapma:270.49768
*/
```

Bir başka örnek olarak 5 elemanlı diziye girilen elemanlardan **tekil** (unique) olanlarını bulan program verilebilir.

```
#include <iostream>
#define BOYUT 5
int main() {
    int dizi[BOYUT];
    int indis;
    for (indis=0; indis<BOYUT; indis++) {
        std::cout << indis << ". Sayıyı Girin:";
        std::cin >> dizi[indis];
    }
    std::cout << "Dizi içinde tekil (unique) olan rakamlar:"<< std::endl;
    for (indis=0; indis<BOYUT; indis++) {
        int indis2, sayac = 0;  // Her elemanda sayacı sıfırla
        for (indis2 = 0; indis2 < BOYUT; indis2++)
        if (indis != indis2) //elemanın kendisini kontrol etmiyoruz
        if (dizi[indis] == dizi[indis2])
        sayac++;
        if (sayac == 0)
        std::cout << "dizi["<< indis << "]:" << dizi[indis] << " tekil olarak dizide bulundu."<<
         ↪  std::endl;
    }
}
/* Program Çıktısı:
0. Sayıyı Girin:23
1. Sayıyı Girin:12
2. Sayıyı Girin:23
3. Sayıyı Girin:14
4. Sayıyı Girin:14
Dizi içinde tekil (unique) olan rakamlar:
dizi[1]:12 tekil olarak dizide bulundu.
*/
```

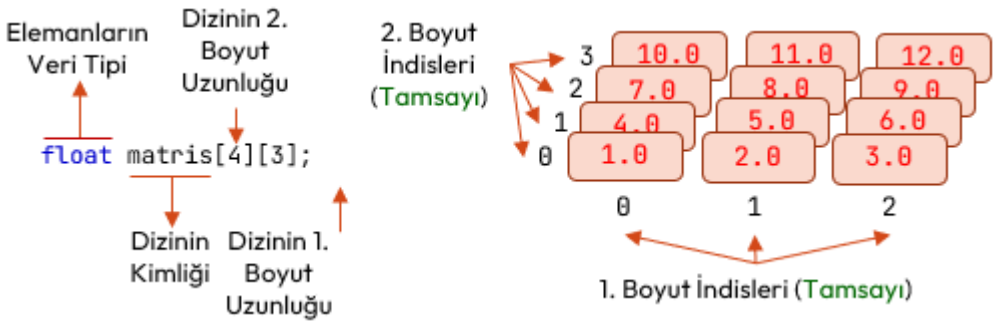
Bir başka örnek olarak 5 elemanlı diziye girilen en büyük elemanına en küçük elemanını ekleyen program verilebilir;

```
#include <iostream>
#include <iomanip> // setw()
#define BOYUT 5
int main() {
    int dizi[BOYUT]={10,12,3,4,6};
    int enBuyuk, enKucuk, enKucukIndex, enBuyukIndex, i ;
    std::cout << "ÖNCE:" << std::endl; //Dizinin İlk hali konsola yazdırılıyor.
    for (i = 0; i < BOYUT; i++)
    std::cout << std::setw(4) << dizi[i];
    std::cout << std::endl;
    enKucuk=dizi[0]; enBuyuk=dizi[0]; /* İlk elemanlar hem en küçük
    hem de en büyük olsun. */
    enKucukIndex=0,enBuyukIndex=0; /* En küçük ve en büyük elemanların
```

```
    indisi 0 olsun. */
    for (i=0; i<BOYUT; i++){ /* En küçük ve en büyük eleman ile
        indislerini bulan kısım. */
        if (dizi[i]<enKucuk) {
            enKucuk=dizi[i];
            enKucukIndex=i;
        }
        if (dizi[i]>enBuyuk) {
            enBuyuk=dizi[i];
            enBuyukIndex=i;
        }
    }
    std::cout << "En Büyük Eleman: dizi[" << enBuyukIndex << "]:"
    << dizi[enBuyukIndex] << std::endl;
    std::cout << "En Küçük Eleman: dizi["<< enKucukIndex << "]: "
    << dizi[enKucukIndex] << std::endl;
    dizi[enBuyukIndex]+=dizi[enKucukIndex]; //En büyük elemana en küçüğünü ekleme
    std::cout << "SONRA:" << std::endl; //Dizinin son hali konsola yazdırılıyor.
    for (i = 0; i < BOYUT; i++)
    std::cout << std::setw(4) << dizi[i];
}
/* Program Çıktısı:
ÖNCE:
10  12   3   4   6
En Büyük Eleman: dizi[1]:12
En Küçük Eleman: dizi[2]: 3
SONRA:
10  15   3   4   6
*/
```

8.3 İki Boyutlu Diziler

Şu ana kadar gördüğümüz tek boyutlu dizilerdi. **İki boyutlu diziler** (two-dimentional array) matematikten de bildiğimiz üzere matris (matrix) olarak adlandırılır. İki boyutlu dediğimizde en-boy ve satır-sütun gibi kavramlar akla gelmelidir. Matris işlemleri gibi bazı problemlerde; bir dizinin her bir elemanının da dizi olması istenir. Bu tür iki boyutlu dizilerde en içteki dizinin boyutu kimliklendirmede sağda yer alır.



Şekil 8.2: İki Boyutlu Dizi Tanımlama

Örneği verilen iki boyutlu dizinin ikinci boyutundan bakacak olursak her biri ayrı bir tek boyutlu dizidir. Bellekte ikici boyutu oluşturan bu diziler de sırasıyla birbirine bitişik (contigious) olarak yer alır. Aslında `float dizi[3][4];` ile `float dizi[12];` aynı bellek yerleşimine sahiptir. Aşağıda iki boyutlu matris tanımlamalarına örnek verilmiştir;

```
float matris[2][3];  //Her bir elemanı 3 elemanlı bir dizi olan 2 satır üç sütunlu bir matris
//Aşağıda 2x3 matrisi başlatma:
float matris2[2][3]= {
    {1.0,2.0,3.0}, //Birinci Satır: 3 Elemanlı bir dizi ile başlatılıyor
    {2.0,4.0,6.0}  //İkinci Satır: 3 Elemanlı bir başka dizi ile başlatılıyor.
};
int karematris[2][2];
//2x2 matrise ilk değer verme:
int karematris2[2][2]= {
```

```
    {1,2}, //Birinci Satır: 2 elemanlı bir dizi
    {5,6} //Birinci Satır: 2 elemanlı bir dizi
};
int karematris3[3][3];
```

Örnek olarak üç satır ve dört sütundan oluşmuş 3x4'lük matris elemanlarını klavyeden girip, tablo halinde ekrana yazdıran programı verebiliriz;

```
#include <iostream>
#include <iomanip> // setw()
#define SATIR 3
#define SUTUN 4
int main() {
    int matris[SATIR][SUTUN];
    int i,j;
    /*
    for (j=0; j<SUTUN; j++)  //Birinci Satır Okuyalım
    std::cin >> matris[0][j];
    for (j=0; j<SUTUN; j++)  //İkinci Satır Okuyalım
    std::cin >> matris[1][j];
    for (j=0; j<SUTUN; j++)  //Üçüncü Satır Okuyalım
    std::cin >> matris[2][j];
    //Bunun yerine iç içe iki for tanımlanır;
    */
    for (i=0; i<SATIR; i++) //İkinci boyut için i indisi
    for (j=0; j<SUTUN; j++)  //Birinci boyut için j indisi
    std::cin >> matris[i][j];
    std::cout <<"TABLO:"<< std::endl;
    for (i=0; i<SATIR; i++) {  //İkinci Boyut İçin
        for (j=0; j<SUTUN; j++)  //Birinci Boyut İçin
        std::cout << std::setw(4) << matris[i][j];
        std::cout << std::endl;
    }
}
/* Program Çıktısı:
1 3 4 5 8 7 6 5 4 4 5 6 8 9 0 2
TABLO:
1   3   4   5
8   7   6   5
4   4   5   6
*/
```

Bir başka örnek olarak elemanları klavyeden girilen 3x2'lik matrisin satır ve sütun toplamlarını ekrana yazan programı verilebilir;

```
#include <iostream>
#define SATIR 3
#define SUTUN 2
int main() {
    int matris[SATIR][SUTUN]; //Matris Tanımı
    int i,j;  //i sutun için, j satır için tanımlandı
    for (i=0; i<SATIR; i++) {  //İkinci Boyut (SATIR) İçin
        std::cout << i << ". Satırı Girin:";
        for (j=0; j<SUTUN; j++)  //Birinci Boyut (SUTUN) İçin
            std::cin >> matris[i][j];
    }
    int satirToplamlar[SATIR]={0}; //Bütün elemanları 0 olan dizi
    for (i=0; i<SATIR; i++) {  // İkinci Boyut İçin
        for (j=0; j<SUTUN; j++) // Birinci Boyut İçin
            satirToplamlar[i]+=matris[i][j];
            /* İçdeki for bitince tüm satırdaki elemanlar toplanmış oldu */
    } /* Dışdaki for ile de her bir satır için toplam tekrarlanıyor */
    std::cout << "Satır Toplamları:" << std::endl;
    for (i=0; i<SATIR; i++)
```

```
        std::cout << i << ". Satır Toplamı:" <<satirToplamlar[i] << std::endl;
    int sutunToplamlar[SUTUN]={0}; //Bütün elemanları 0 olan dizi
    for (j=0; j<SUTUN; j++) // Birinci Boyut İçin
        for (i=0; i<SATIR; i++) {  // İkinci Boyut İçin
            sutunToplamlar[j]+=matris[i][j];
            /* İçdeki for bitince tüm sutundaki elemanlar toplanmış oldu*/
        }/* Dışdaki for ile de her bir sutun için toplam tekrarlanıyor */
    std::cout << "Sütun Toplamları:" << std::endl;
    for (j=0; j<SUTUN; j++)
        std::cout << j << ". Sütun Toplamı:" << sutunToplamlar[j] << std::endl;
}
/* Program Çıktısı:
0. Satırı Girin:2 8
1. Satırı Girin:13 17
2. Satırı Girin:16 34
Satır Toplamları:
0. Satır Toplamı:10
1. Satır Toplamı:30
2. Satır Toplamı:50
Sütun Toplamları:
0. Sütun Toplamı:31
1. Sütun Toplamı:59
*/
```

8.4 Çok Boyutlu Diziler

Şu ana kadar gördüğümüz tek boyutlu diziler veya iki boyutlu diziler olan matrislerdir. Çok boyutlu dizilere örnek olarak En-Boy-Yükseklik içeren 3 boyutlu diziler verilebilir. Üç boyutlu dizilerde dizinin her bir elemanı bir matristir. Çok boyutlu dizilerde de en içteki dizinin boyutu kimliklendirmede sağda yer alır. Aşağıdaki örnekte; çeşitli boyutlarda diziler tanımlanmıştır;

```
int ucbuyutludizi1[3][2][2]; // ucbuyutludizi1: elemanları 3 adet 2x2 matris olan üç boyutlu bir
↪ dizi
int ucbuyutludizi1[2][3][3]; // ucbuyutludizi2: elemanları 2 adet 3x3 matris olan üç boyutlu bir
↪ dizi
int ucbuyutludizi1[4][3][2]; // ucbuyutludizi3: elemanları 4 adet 3x2 matris olan üç boyutlu bir
↪ dizi
int dortbuyutludizi1[5][2][3][2]; /* dortbuyutludizi1: elemanları 5 adet olan ve her bir elemanı
↪ 2 adet 3x2 matris olan dört boyutlu bir dizi */
```

Aşağıda üç boyutlu dizilerde bir ilk değer verme kod örneği verilmiştir;

```
#define DERINLIK 3
#define SATIR 3
#define SUTUN 3
int ucBoyutluDizi[DERINLIK][SATIR][SUTUN]={
    { {1,2,3},{4,5,6},{7,8,9} },
    { {11,12,13},{14,15,16},{17,18,19} },
    { {21,22,23},{24,25,26},{27,28,29} }
};
```

Çok boyutlu dizi örneği olarak şöyle bir örnek verilebilir; 3 farklı ders alan 10 öğrenci, her bir dersten 4 değişik not almaktadır. Bu notların ağırlıkları sırasıyla %10, %30, %20 ve %40'tır. Notlar rastgele 0 ile 100 arasında verilecektir. Her bir sınavın ortalaması ile her bir öğrencinin ağırlıklı not ortalamalarını bulan C++ programı aşağıdaki gibidir;

```
#include <iostream>
#include <vector>
#include <random> // Modern rastgele sayı üretimi için
#include <iomanip> // Çıktı formatlama (F2, F1 gibi) için
int main() {
    // Sabit değerlerin tanımlanması
    const int DERS_SAYISI = 3;
    const int SINAV_SAYISI = 4;
    const int OGRENCI_SAYISI = 10;
```

```
    // C++ Çok Boyutlu Dizi Tanımı
    int notlar[DERS_SAYISI][SINAV_SAYISI][OGRENCI_SAYISI];
    int agirlik[] = { 10, 30, 20, 40 };
    // Modern C++ Rastgele Sayı Üreticisi
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> dis(0, 100);
    // 1. Notları Rastgele Doldurma
    for (int i = 0; i < DERS_SAYISI; i++) {
        for (int j = 0; j < SINAV_SAYISI; j++) {
            for (int k = 0; k < OGRENCI_SAYISI; k++) {
                notlar[i][j][k] = dis(gen);
            }
        }
    }
    // 2. Sınav Ortalamalarını Hesaplama
    std::cout << "Sinav Ortalamalari:\n";
    for (int i = 0; i < DERS_SAYISI; i++) {
        std::cout << i << ". Ders:\n";
        for (int j = 0; j < SINAV_SAYISI; j++) {
            std::cout << " " << j << ". Sinav: ";
            int notToplam = 0;
            for (int k = 0; k < OGRENCI_SAYISI; k++) {
                notToplam += notlar[i][j][k];
            }
            float ortalama = static_cast<float>(notToplam) / OGRENCI_SAYISI;
            std::cout << std::fixed << std::setprecision(2) << ortalama << " ";
        }
        std::cout << "\n";
    }
    // 3. Her bir Öğrencinin Ağırlıklı Ortalaması
    std::cout << "\nHer bir Ogrencinin Agirlikli Ortalamasi:\n";
    for (int k = 0; k < OGRENCI_SAYISI; k++) {
        std::cout << k << ". Ogrenci: ";
        for (int i = 0; i < DERS_SAYISI; i++) {
            std::cout << " " << i << ". Ders: ";
            float agirlikliNot = 0.0f;
            for (int j = 0; j < SINAV_SAYISI; j++) {
                agirlikliNot += (notlar[i][j][k] * agirlik[j]) / 100.0f;
            }
            std::cout << std::fixed << std::setprecision(1) << agirlikliNot << " ";
        }
        std::cout << "\n";
    }
}
/*Program Çıktısı:
Sinav Ortalamalari:
0. Ders:
0. Sinav: 51.00  1. Sinav: 46.30  2. Sinav: 47.50  3. Sinav: 35.30
1. Ders:
0. Sinav: 61.10  1. Sinav: 60.00  2. Sinav: 74.30  3. Sinav: 40.80
2. Ders:
0. Sinav: 42.10  1. Sinav: 31.60  2. Sinav: 56.90  3. Sinav: 43.70

Her bir Ogrencinin Agirlikli Ortalamasi:
0. Ogrenci:  0. Ders: 68.2  1. Ders: 42.7  2. Ders: 66.4
1. Ogrenci:  0. Ders: 40.2  1. Ders: 51.3  2. Ders: 18.0
2. Ogrenci:  0. Ders: 34.0  1. Ders: 68.8  2. Ders: 55.5
3. Ogrenci:  0. Ders: 29.8  1. Ders: 72.6  2. Ders: 48.2
4. Ogrenci:  0. Ders: 42.3  1. Ders: 75.5  2. Ders: 42.2
5. Ogrenci:  0. Ders: 35.9  1. Ders: 30.6  2. Ders: 35.8
6. Ogrenci:  0. Ders: 7.7  1. Ders: 46.1  2. Ders: 49.6
```

```
7. Ogrenci:  0. Ders: 47.9  1. Ders: 36.0  2. Ders: 29.7
8. Ogrenci:  0. Ders: 72.3  1. Ders: 59.0  2. Ders: 51.4
9. Ogrenci:  0. Ders: 47.8  1. Ders: 70.3  2. Ders: 28.7
*/
```

8.5 Parametre Olarak Diziler

Parametre olarak gönderilen dizinin boyutu kadar köşeli parantez açılır ve kapatılır, ilk köşeli parantez içerisine eleman sayısını ifade eden değer yazılmaz, fakat diğerlerine eleman sayıları verilmek zorundadır. Bu durumda tek boyutlu diziler parametre olacak ise dizinin boyutu yazılmayabilir. Aşağıda dizileri parametre olarak alan fonksiyon bildirim örnekleri bulunmaktadır;

```
void diziIsleyenFonksiyon1(int tekBoyutluDizi[],int boyut);
void diziIsleyenFonksiyon2(int pDizi[][2],int ikinciBoyut,int birinciBoyut);
void diziIsleyenFonksiyon3(int pDizi[][3][2],int ucuncuBoyut,int ikinciBoyut,int birinciBoyut);

// Bu fonksiyonları aşağıdaki şekilde çağırabiliriz;
unsigned notlar[10]; //10 öğrenci notu barındırır.
unsigned dersNotlari[2][10]; //2 dersi alan 10 öğrenci için notlar;
unsigned bolumDersNotlari[4][2][10]; /* 4 bölümde 2 dersi alan 10 öğrenci için notlar;*/
//...
diziIsleyenFonksiyon1(notlar,10);
diziIsleyenFonksiyon2(dersNotlari,2,10);
diziIsleyenFonksiyon3(bolumDersNotlari,4,2,10);
//...
```

Aşağıda bir diziyi okuyan, ekrana yazan ve eleman ortalamalarını hesaplayan fonksiyonlara sahip bir program verilmiştir;

```
#include <iostream>
#include <iomanip> // setw()
void diziOku(int pDizi[],int pUzunluk);
void diziYaz(int pDizi[],int pUzunluk);
float diziOrtalama(int pDizi[],int pUzunluk);
int main() {
    int dizi[5];
    float ortalama;
    diziOku(dizi,5); /* fonsiyon geri dönüş değeri olmadığından bir değişkene atanmıyor! */
    diziYaz(dizi,5); /* fonsiyon geri dönüş değeri olmadığından bir değişkene atanmıyor! */
    ortalama=diziOrtalama(dizi,5); /* fonksiyon geri dönüş değeri bir değişkene atanıyor.
     ↪ Atandığı değişkenin tipi ile fonksiyonun geri dönüş tipi aynı olmalı! */
    std::cout << "Dizi Ortalaması:" <<ortalama ;
    return 0;
}
void diziOku(int pDizi[],int pUzunluk){
    int sayac;
    std::cout << pUzunluk << " Elemanlı Dizi Okunacaktır:";
    for (sayac=0; sayac < pUzunluk; sayac++)
        std::cin >> pDizi[sayac];
}
void diziYaz(int pDizi[],int pUzunluk){
    int sayac;
    std::cout << pUzunluk << " Elemanlı Dizi:" << std::endl;
    for (sayac=0; sayac < pUzunluk; sayac++)
        std::cout << std::setw(5) << pDizi[sayac] ;
    std::cout << std::endl;
}
float diziOrtalama(int pDizi[],int pUzunluk){
    int sayac;
    float toplam=0;
    for (sayac=0; sayac < pUzunluk; sayac++)
        toplam+=pDizi[sayac];
    return toplam/pUzunluk;
}
```

```
/* Program Çıktısı:
5 Elemanlı Dizi Okunacaktır:10 20 30 40 50
5 Elemanlı Dizi:
10   20   30   40   50
Dizi Ortalaması:30
*/
```

8.6 Aralık Tabanlı For Döngüsü

Yalnızca bir dizideki öğeler arasında döngü oluşturmak için kullanılan döngü, **her bir eleman için döngü** (foreach loop) yada **aralık tabanlı döngü** (range based looprange based loop) olarak da bilinir. Bu döngü ileride göreceğimiz Konteyner Şablonları başlığındaki veri yapılarıyla da kullanılır. Aşağıdaki gibi yazılır;

```
for (veritipi dizi-elemanını-temsil-eden-değişken : dizi-kimliği)
    döngü-kodu;
```

Buna örnek olarak aşağıda bir dizi programı örneği verilebilir;

```
#include <iostream>
#include <iomanip> // setw()
int main() {
    int dizi[5]={2,4,6,8,10};
    std::cout << "Dizi:" << std::endl;
    for (int eleman: dizi)
        std::cout << std::setw(5) << eleman ;
    std::cout << std::endl;
    float toplam=0;
    for (int eleman: dizi)
        toplam+=eleman;
    std::cout << "Dizi Ortalaması:" <<toplam /(sizeof (dizi)/sizeof (int));
}
/* Program Çıktısı:
Dizi:
2    4    6    8   10
Dizi Ortalaması:6
*/
```

8.7 Düşün ve Çöz

- **Düşün**: Çok boyutlu dizilerin (matrislerin) bellekte aslında tek boyutlu ve ardışık olarak tutulmasının, dizi elemanlarına erişim hızına etkisini tartışınız.
- **Düşün**: C++ dilinde dizi sınırlarının (out of bounds) kontrol edilmemesinin, bellek güvenliği ve ara bellek taşması (buffer overflow) saldırıları açısından riskini tartışınız.
- **Çöz**: 10 elemanlı rastgele sayılardan oluşan bir diziyi kabarcık sıralama (bubble sort) algoritması ile küçükten büyüğe sıralayan fonksiyonu yazınız.
- **Çöz**: 20 elemanlı bir dizideki elemanların frekansını (hangi sayıdan kaç tane olduğunu) bulan ve ekrana listeleyen programı yazınız.
- **Çöz**: Kullanıcıdan alınan iki adet 3x3'lük matrisin toplamını bulan ve sonucu ekrana matris formunda basan fonksiyonu kodlayınız.
- **Çöz**: İki boyutlu bir diziyi bellekte satır öncelikli (row-major) düzende saklandığını düşünerek, `int matris[3][4];` dizisinin `matris[1][2]` elemanının bellek adresini hesaplayın (başlangıç adresi 1000, her int 4 bayt).
- **Düşün**: Aralık tabanlı for (range-based for) döngüsünü klasik indis tabanlı döngüyle karşılaştırın. Aralık tabanlı for döngüsünün avantajları ve sınırlılıkları nelerdir? Hangi durumda klasik döngü zorunlu hale gelir?
- **Düşün**: Bir diziyi fonksiyona parametre olarak geçirdiğinizde C++'da ne olur? Fonksiyon içinde dizinin boyutunu `sizeof` ile öğrenebilir misiniz? Bu **dizi bozunması** (array decay) sorununu nasıl çözersiniz?
- **Çöz**: Çok boyutlu dizi ile işaretçiler dizisi arasındaki farkı açıklayın. `int mat[3][4];` ile `int* mat[3];` ifadelerinin bellekte nasıl farklı yerleştiğini gösterin.